Session - 5

Organization : TeachToTech
Instructor : Ayush Raj
Duration : 1.5 hr

(knight @leetcode, 4★@codechef, Specialist @codeforces)

CHAPTER 1

Functions & Recursion :

What is a Function?

The Coffee Machine Analogy

Imagine making coffee 5 times daily. Do you grind beans, boil water, and filter manually each time? No! You buy a machine and press a button.

In coding: A function is that machine. Build it once, call it whenever needed.

Reusability

Write once, use everywhere in your code

Efficiency

Save time and reduce repetitive coding

Organization

Keep your code clean and maintainable

Anatomy of a Function

Understanding the three essential components of every function

Declaration

The Menu - tells the compiler a function is coming later

Definition

The Recipe - the actual code inside the curly braces

Call

The Order - using the function in main()

```
// 1. Declaration
int add(int a, int b);

int main() {
 // 3. Call
 int sum = add(5, 10);
}

// 2. Definition
int add(int a, int b) {
 return a + b;
}
```

Pass by Value vs. Reference

The Photocopy Analogy

Pass by Value

Sharing notes by giving a **photocopy**. If you scribble on the copy, does my original change? No.

In C: Sending a variable sends a *copy*. Changes inside the function don't affect the original.

Pass by Reference

Giving you my **original notebook**. If you scribble on it, my notes are ruined.

In C: We use addresses (&) and pointers (*) to touch the original data directly.

CHAPTER 2

Introduction to Recursion

Definition

When a function calls *itself*

The Golden Rule

Every recursion MUST have a **Base Case** (stop condition). Without it, the function calls itself forever → Stack Overflow → Crash!

Analogy

Russian Nesting Dolls. Open one to find a smaller one, until you reach the tiny solid one (the base case).

Problem 1: Print 1 to N

Using Recursion Without Loops

01

Base Case

If n == 0, stop

02

Recursive Step

Call print(n-1) first

03

Work

Print n after the call returns

Visual Trace Example

- print(3) calls print(2)
- print(2) calls print(1)
- print(1) calls print(0) → STOP
- print(1) resumes → Prints 1
- print(2) resumes → Prints 2
- print(3) resumes → Prints 3

Recursion (Problem Set - 1)

LeetCode

Reverse Integer – LeetCode

Can you solve this real interview question? Reverse Integer – Given a...

LeetCode

Fibonacci Number – LeetCode

Can you solve this real interview question? Fibonacci Number – The...

LeetCode

Reverse String – LeetCode

Can you solve this real interview question? Reverse String – Write a...

Activity: Factorial

The Classic Recursion Problem

The Math

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

Recursive Formula

$$5! = 5 \times 4!$$

Each factorial depends on the previous one, creating a natural recursive pattern.

The Code

```
int fact(int n) {
 if (n == 0)
 return 1; // Base Case

 return n * fact(n - 1); // Recursive Call
}
```

Problem 2: Fibonacci Sequence

The Tree Structure

Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21...

Formula

fib(n) = fib(n-1) + fib(n-2)

The Challenge

It calls itself twice, creating a tree structure

Base Case

If n is 0 or 1, return n

Problem 3: Reverse Integer/String

Integer Reversal Logic

- Pass n and a sum (store result)
- sum = sum * 10 + n % 10
- Call function with n / 10

String Reversal Logic

- Swap first and last characters
- Recursively call function for middle part
- Continue until base case reached

Key Takeaways

1

Functions are Reusable

Write once, use everywhere - like a coffee machine for your code

2

Understand Pass Methods

Value creates copies, reference touches originals

3

Recursion Needs Base Cases

Always define when to stop, or face stack overflow

4

Practice Makes Perfect

Master factorial, Fibonacci, and reversal problems